

ERP Internship Project Master Architecture Document

Enterprise Resource Planning (ERP) System – Engineering Blueprint

1. Document Purpose

This document defines the **architecture, engineering rules, delivery workflow, team responsibilities, service boundaries, repository structure, and implementation standards** for the internship ERP project.

The goal is to ensure that **25 interns working in 5 teams** can build a **single ERP system** without breaking each other's work, duplicating logic, or creating incompatible backend/frontend implementations.

This ERP project will be built using:

- **Backend:** Java, Spring Boot, Spring Security, Spring Data JPA
 - **Frontend:** Next.js, React, TypeScript
 - **Database:** PostgreSQL
 - **Messaging / Event-driven integration:** RabbitMQ
 - **Architecture style:** Domain-based microservices with shared frontend
 - **Version control:** Git + Pull Request workflow
 - **Containerization:** Docker / Docker Compose
-

2. Project Overview

The ERP system consists of **five core business modules**, each implemented as an independent Spring Boot microservice, while all modules share a **single Next.js frontend application**.

ERP Modules

Module 1: Human Resource Management System (HRM)

Handles employees, departments, attendance, leave, payroll-related HR operations, and organizational structure.

Module 2: Procurement and Resource Management System (PRMS)

Handles suppliers, purchase requests, purchase orders, approvals, and procurement workflows.

Module 3: Material Management System (MMS)

Handles item master data, warehouses, stock movement, goods receipt, goods issue, and inventory control.

Module 4: Sales & Customer Relationship Management (CRM)

Handles customers, leads, quotations, sales orders, and customer interactions.

Module 5: Finance Management System (FMS)

Handles chart of accounts, journals, accounts payable/receivable, invoices, payments, and financial reporting.

3. Team Structure

There are **25 interns**, divided into **5 teams**, with **5 members per team**.

Team-to-Module Mapping

- **Team 1** → HRM
- **Team 2** → PRMS
- **Team 3** → MMS
- **Team 4** → CRM
- **Team 5** → FMS

Each team is the **primary owner** of its module's backend service and frontend feature area.

4. Team Internal Role Distribution

Each team should split work using the following role structure.

Recommended team roles

1. Team Lead / Module Coordinator

Responsible for:

- module planning
- sprint task breakdown
- pull request review coordination
- service contract discussions
- cross-team communication

2. Backend Engineer A

Responsible for:

- entity modeling
- repository implementation
- core CRUD endpoints
- validation rules

3. Backend Engineer B

Responsible for:

- business workflows
- event publishing/consuming
- service integrations
- automated tests

4. Frontend Engineer

Responsible for:

- pages and routes for the module
- forms, tables, dashboards
- API integration in Next.js
- module UI consistency

5. QA / Integration / Support Engineer

Responsible for:

- module testing support
- Postman/API validation
- integration verification
- bug reporting
- release/demo support

Note: Team members can still collaborate across tasks, but **ownership must be explicit**. If nobody owns integration, testing, or frontend coordination, those areas get ignored.

5. High-Level System Architecture

The ERP system is composed of:

- **1 shared frontend application** built with Next.js
- **5 backend microservices** built with Spring Boot

- **1 PostgreSQL instance** with isolated schemas per service
- **1 RabbitMQ broker** for event-driven communication
- shared development infrastructure using Docker Compose

Core architecture components

Frontend

A single **Next.js ERP web application** that contains:

- authentication pages
- shared dashboard shell
- shared navigation
- modulespecific routes and pages for HRM, PRMS, MMS, CRM, and FMS

Backend microservices

Five domain-aligned services:

- hrm-service
- prms-service
- mms-service
- crm-service
- fms-service

Data layer

One PostgreSQL server with **separate schema ownership per service**.

Event layer

RabbitMQ is used for:

- domain event publishing
- cross-service notifications
- asynchronous integration workflows

6. Architecture Principles

The entire ERP must follow these principles.

6.1 Domain ownership

Each service owns one business domain and is responsible for its own:

- entities
- business rules
- API endpoints
- database schema
- events
- tests

6.2 Loose coupling

Services must not directly modify each other's database tables or schema.

All interservice communication must happen through:

- REST APIs, or
- RabbitMQ events

6.3 Shared frontend, isolated feature ownership

The frontend is a single Next.js project, but each team owns only its **feature area** inside the shared frontend.

6.4 Contract-first collaboration

Teams must define and agree on:

- endpoint contracts
 - event payload contracts
 - permission naming
 - frontend route ownership
- before heavy implementation begins.

6.5 Production-style engineering discipline

This is not just a CRUD internship exercise. Every module must include:

- validation
- security
- audit fields
- tests
- API documentation
- error handling
- migration scripts
- integration considerations

7. Backend Service Architecture

7.1 Required backend services

The ERP backend must be implemented as the following Spring Boot services:

1. HRM Service

Responsible for employee and workforce management.

2. PRMS Service

Responsible for procurement and supplier workflows.

3. MMS Service

Responsible for materials and inventory workflows.

4. CRM Service

Responsible for customer and sales workflows.

5. FMS Service

Responsible for finance and accounting workflows.

8. Service Ownership Boundaries

This section defines **what each service owns**. This is one of the most important parts of the architecture.

8.1 HRM Service Boundary

HRM owns:

- employee master data
- departments
- positions / job titles
- attendance
- leave requests
- organizational structure

- employee status (active, inactive, terminated)
- payroll-related HR source data

HRM must not own:

- accounting journals
- purchase orders
- stock balances
- customer records

HRM may integrate with:

- FMS for payroll or salary expense posting
 - shared auth for user-role mapping if needed
-

8.2 PRMS Service Boundary

PRMS owns:

- suppliers
- purchase requests
- purchase approvals
- purchase orders
- procurement workflows
- procurement status tracking

PRMS must not own:

- stock balances
- journal posting
- employee records
- customer sales records

PRMS may integrate with:

- MMS for goods receiving or inventory-linked procurement
 - FMS for supplier payable / procurement financial impact
-

8.3 MMS Service Boundary

MMS owns:

- item master data
- warehouse records
- stock balances
- stock movements
- goods receipt
- goods issue
- inventory adjustments
- reorder tracking

MMS must not own:

- purchase approval workflow
- finance journal ownership
- employee attendance
- customer invoicing

MMS may integrate with:

- PRMS for approved purchase orders / receiving
 - CRM for sales-related stock movement
 - FMS for inventory valuation/accounting impact
-

8.4 CRM Service Boundary

CRM owns:

- customers
- leads
- opportunities
- quotations
- sales orders
- customer communications or status tracking

CRM must not own:

- stock ledger ownership
- journal ownership
- payroll ownership
- supplier procurement workflow

CRM may integrate with:

- MMS for inventory availability or stock issue
 - FMS for invoices, receivables, and payment status
-

8.5 FMS Service Boundary

FMS owns:

- chart of accounts
- journals
- accounts payable
- accounts receivable
- invoices
- payments
- budgeting
- financial reports
- posting and accounting rules

FMS must not own:

- employee attendance details
- supplier procurement workflow
- stock receiving process logic
- customer lead management

FMS may consume events from:

- HRM
- PRMS
- MMS
- CRM

FMS is the most integration-heavy service because finance depends on business events from other modules.

9. Cross-Service Communication Rules

To prevent service coupling and broken ownership, the following rules are mandatory.

Rule 1: No direct database access across services

A service must **never** write to another service's tables or schema.

Not allowed:

- PRMS writing directly into MMS stock tables
- CRM directly inserting finance invoice rows
- FMS directly updating HRM employee records

Rule 2: Cross-service communication must use one of two approved mechanisms

A. REST API

Use synchronous service-to-service communication when a service needs immediate data.

Example:

- PRMS requests item information from MMS
- FMS requests payroll summary data from HRM

B. RabbitMQ events

Use asynchronous communication when a business event happens and other services may react to it.

Example:

- PurchaseOrderApproved
- SalesOrderConfirmed
- PayrollProcessed
- StockReceived
- InvoiceGenerated

10. Event-Driven Integration Strategy

RabbitMQ is the backbone for asynchronous ERP integration.

10.1 Why use events?

Events allow one module to notify others without tightly coupling the systems.

Example:

- PRMS approves a purchase order
- MMS consumes the event to prepare goods receiving
- FMS consumes the event to create payable/expense-related financial records

10.2 Event naming convention

Events must use clear business names in **past-tense business-event format**.

Examples

- EmployeeCreated
- EmployeeUpdated
- LeaveApproved
- PurchaseRequestCreated
- PurchaseOrderApproved
- StockReceived
- InventoryAdjusted
- CustomerCreated
- SalesOrderConfirmed
- InvoiceGenerated
- PaymentRecorded
- JournalPosted

10.3 Event payload rules

Every event payload should include:

- event name
- event ID
- timestamp
- source service
- entity/business record ID
- business data required by consumers

Example event payload

```
{
  "eventId": "uuid",
  "eventType": "PurchaseOrderApproved",
  "occurredAt": "2026-07-08T12:00:00Z",
  "source": "prms-service",
  "data": {
    "purchaseOrderId": 102,
    "supplierId": 7,
```

```
    "totalAmount": 150000,  
    "currency": "ETB"  
  }  
}
```

10.4 Event contract ownership

The producing service owns the event contract, but consumers must review it before final approval.

11. Database Architecture

11.1 Database strategy

The ERP system will use **one PostgreSQL server** with **separate schemas per service**.

Schemas

- hrm_schema
- prms_schema
- mms_schema
- crm_schema
- fms_schema

Why separate schemas?

This gives:

- service isolation
- easier ownership boundaries
- lower infrastructure complexity than separate DB servers
- clear migration responsibility

11.2 Database ownership rule

Each team owns:

- only its schema
- only its tables
- only its migration scripts
- only its seed data

No team may manually modify another team's schema.

11.3 Migration tool

All schema changes must be managed with **Flyway** migration scripts.

Rules

- every schema change must be versioned
 - no direct manual production-like schema edits
 - migration files must be committed to Git
 - schema changes must be reviewed in pull requests
-

12. Security Architecture

The ERP system must use a shared security model.

12.1 Authentication

Authentication should be handled using:

- Spring Security on the backend
- JWT or centralized auth integration
- protected frontend routes in Next.js

12.2 Authorization

The system must use **Role-Based Access Control (RBAC)** and permission-based authorization.

Example roles

- Super Admin
- HR Manager
- Procurement Officer
- Inventory Manager
- Sales Officer
- Finance Officer
- Auditor
- Employee

Example permission naming

- `hrm.employee.read`
- `hrm.employee.create`
- `hrm.leave.approve`
- `prms.purchase_order.approve`

- `mms.stock.adjust`
- `crm.customer.create`
- `fms.journal.post`

12.3 Security rules

- all write endpoints must be protected
 - sensitive read endpoints must be permission-checked
 - frontend menus must be permission-aware
 - audit-related operations must identify the acting user where possible
-

13. Shared Frontend Architecture

The ERP frontend is a **single shared Next.js application**, but it must be organized in a way that allows five teams to work in parallel safely.

13.1 Frontend architecture goals

- one consistent ERP UI
- one login/auth flow
- one dashboard shell
- module isolation by feature folder
- minimal merge conflicts between teams
- clear ownership boundaries

13.2 Frontend structure model

The frontend will have **three layers**:

A. Shared platform layer

Contains global application infrastructure:

- auth handling
- route protection
- shared layout
- sidebar and navigation
- shared API client
- permission utilities
- global error handling
- shared UI infrastructure

B. Shared component library

Contains reusable components:

- tables
- forms
- modals
- inputs
- badges
- status chips
- cards
- chart wrappers

C. Module feature areas

Each team owns its module feature folder:

- HRM feature folder
- PRMS feature folder
- MMS feature folder
- CRM feature folder
- FMS feature folder

14. Frontend Ownership Rules

Shared frontend code that must be reviewed carefully

The following areas are shared and should only be modified through careful review:

- app shell / layout
- sidebar navigation config
- auth middleware
- shared UI library
- API client infrastructure
- shared route guards
- permission utilities
- theme / design tokens
- global state management if used

Team-owned frontend areas

Each team owns only its module's feature folder and module routes.

HRM team owns:

- HRM pages
- HRM forms
- HRM API hooks
- HRM UI components

PRMS team owns:

- PRMS pages
- PRMS tables
- PRMS workflows
- PRMS API integration

MMS team owns:

- MMS inventory pages
- MMS stock movement pages
- MMS forms and data tables

CRM team owns:

- CRM customer pages
- CRM order pages
- CRM dashboards and forms

FMS team owns:

- FMS journal pages
- FMS invoice pages
- FMS finance reports and tables

15. Backend Code Structure Standard

All Spring Boot services must use a **consistent internal structure**.

Recommended service structure

Each service should be organized using **feature-first packaging**, not a giant global `controller/` `service/repository` dump.

Example structure

- `shared/`
- `config`
- `security`

- exception
- util
- employee/
- controller
- dto
- entity
- repository
- service
- mapper
- event
- leave/
- attendance/
- department/

This approach allows multiple interns to work on different features in the same service without constantly colliding.

16. API Standards

All services must expose APIs using a consistent style.

16.1 URL versioning

All endpoints should use `/api/v1/...`

16.2 Resource naming

Use plural resource naming where appropriate.

Example

- GET `/api/v1/employees`
- POST `/api/v1/employees`
- GET `/api/v1/purchase-orders`
- POST `/api/v1/customers`

16.3 Response format

All services should return a consistent response structure for success and error cases.

Success response example

```
{
  "success": true,
  "message": "Employee created successfully",
  "data": {
    "id": 1,
    "name": "John Doe"
  },
  "timestamp": "2026-07-08T12:00:00Z"
}
```

Error response example

```
{
  "success": false,
  "message": "Validation failed",
  "errors": [
    {
      "field": "email",
      "message": "Invalid email format"
    }
  ],
  "timestamp": "2026-07-08T12:00:00Z"
}
```

16.4 API documentation

Every service must expose Swagger/OpenAPI documentation.

17. Audit and Operational Standards

ERP systems require traceability. Every service must support basic operational controls.

17.1 Audit fields

All major business tables should include:

- created_at
- created_by
- updated_at
- updated_by

Where relevant, business entities should also include:

- status
- approval status
- approval date
- approved by

17.2 Logging

Every service must implement structured logging for:

- incoming requests
- validation failures
- major business actions
- integration errors
- event publishing / event consumption failures

17.3 Exception handling

Each service must have a centralized global exception handler for:

- validation errors
 - not found errors
 - access denied errors
 - business rule violations
 - unexpected system errors
-

18. Development Workflow

18.1 Branching strategy

The project will use:

- `main` → stable branch
- `develop` → integration branch
- `feature/*` → feature branches

Example feature branches

- `feature/hrm-employee-management`
- `feature/prms-purchase-order`
- `feature/mms-stock-receive`
- `feature/crm-sales-order`
- `feature/fms-journal-posting`

18.2 Pull request rules

Every feature must be merged using a pull request.

Each PR must include:

- feature summary
- affected module/service
- DB migration notes
- API changes
- event changes
- test status
- screenshots if frontend is included

18.3 Merge rules

A pull request may be merged only if:

- code review is completed
 - build passes
 - tests pass
 - the change does not break shared contracts
 - documentation is updated if contracts changed
-

19. Definition of Done

A feature is considered complete only if all required engineering work is finished.

A feature is “Done” only when:

- backend endpoint(s) are implemented
 - validation is added
 - DB migration is added if needed
 - frontend integration is completed if applicable
 - permissions are enforced
 - tests are written
 - Swagger docs are updated
 - event publication/consumption is added if required
 - PR review is completed
 - feature is merged without breaking other modules
-

20. Shared Project Repository Strategy

The ERP system will be developed in a **single monorepo**.

Why monorepo?

For an internship project, a monorepo provides:

- easier onboarding
 - centralized setup
 - simpler infrastructure management
 - shared documentation in one place
 - easier code review and visibility
 - easier Docker Compose integration
-

21. High-Level Repository Structure

The repository will contain:

- documentation
- infrastructure setup
- frontend project
- all backend services
- project-level scripts and guides

Repository sections

- /docs
 - /infrastructure
 - /frontend
 - /services
 - project root configuration and README
-

22. Weekly Delivery Model

The internship project will be delivered in **phases**, not random feature hacking.

Phase 0 – Architecture alignment

- team assignment
- standards finalization

- ownership definition
- module scoping

Phase 1 – Domain and contract design

- ERDs
- API contracts
- event contracts
- route/page planning

Phase 2 – Shared skeleton setup

- service skeletons
- frontend shell
- auth skeleton
- Docker setup
- DB schemas and migrations

Phase 3 – Module MVP implementation

- core business entities
- workflows
- basic frontend pages
- CRUD and approval flows

Phase 4 – Cross-service integration

- RabbitMQ events
- service-to-service API calls
- finance and stock integration flows

Phase 5 – QA, hardening, and demo preparation

- bug fixes
- testing
- documentation
- deployment and final demo readiness

23. Governance and Coordination

This project cannot be run as five disconnected teams with zero coordination. There must be a lightweight governance layer.

Required coordination structure

Weekly Team Standup

Each team reports:

- progress
- blockers
- contract changes
- frontend/backend integration issues

Weekly Architecture Sync

Team leads meet with the mentor/project coordinator to review:

- cross-team dependencies
- event contract changes
- shared frontend conflicts
- integration blockers

Weekly Demo

Every team must show something working every week:

- backend endpoint
- frontend flow
- integration demo
- event flow
- test coverage improvement

Slides are not enough. The review should be based on running software.

24. Risks This Architecture Is Designed to Prevent

This architecture exists to prevent the most common internship-project disasters.

Risk 1: Teams breaking each other's work

Solved by:

- service ownership
- schema ownership
- frontend feature ownership

Risk 2: Inconsistent API and frontend patterns

Solved by:

- shared standards
- contract-first design
- shared response/error format

Risk 3: Shared frontend merge chaos

Solved by:

- feature folder ownership
- restricted shared app-shell changes
- clear route ownership

Risk 4: Late integration failure

Solved by:

- early event/API contract definition
- explicit integration phase
- weekly architecture sync

Risk 5: “It works on my machine” syndrome

Solved by:

- Docker Compose
- shared repo
- shared setup guide
- versioned migrations

25. Final Engineering Rules

The following rules are non-negotiable for the ERP project.

Rule 1

A team may freely build inside its module boundary, but must not break another module's contracts.

Rule 2

No service may directly write to another service's schema or tables.

Rule 3

Any shared frontend changes must be reviewed carefully.

Rule 4

Any API contract change or event payload change must be communicated before merging.

Rule 5

All schema changes must go through migration files.

Rule 6

All critical business actions must be permission-aware and traceable.

Rule 7

The project must be built incrementally with weekly demos and integration checkpoints.

26. Conclusion

This ERP project is not just about “building five modules.” It is about learning how to build a **single business system using real engineering discipline**:

- service ownership
- domain boundaries
- shared frontend collaboration
- contract-based integration
- event-driven communication
- secure and traceable workflows
- team coordination without chaos

If the teams follow this architecture document, they will be able to:

- work in parallel safely
- build a coherent ERP platform
- integrate their modules cleanly
- avoid breaking each other's services

- deliver something much closer to a real production engineering project than a typical internship demo

This document is the foundation.

The next step is to define the **detailed blueprint for each module**: HRM, PRMS, MMS, CRM, and FMS — including features, sub-features, entities, endpoints, events, UI pages, and integration points.